The following was generated using ChatGPT and further edited by Lucas Hartman

Chapter 3 – Object-Oriented Programming in Python

Up to this point, you've been writing programs made up of variables, loops, and functions that work together to process data. In this chapter, we'll take a major step forward and look at object-oriented programming (OOP) — a design approach that organizes code around objects rather than just actions or logic. Objects combine both data and behavior, making them powerful tools for building structured and reusable programs.

Python makes it easy to work with objects using classes, which act as templates for creating individual objects. You'll learn how to define classes, create and control objects, hide internal details, and extend behavior through Python's built-in 'magic methods.' OOP is one of the most important topics in programming — and by the end of this chapter, you'll be ready to design and build your own class-based systems.

1. Basics of Classes in Python

A class defines the blueprint of an object — describing what data it holds and what actions it can perform. An object (also called an instance) is a specific realization of that class.

1.1 Create a class

To create a class we use the '`class`' keyword, name it using `CamelCase`, followed by a colon (`:`). We are then ready to code all the functionality and data members that are tabbed in the lines that follow.

```
class Car:
    pass
```

1.2 The Constructor and instance variables

The constructor is a special method used to create the object. In python it is defined by `__init__()`.

The constructor initializes data when an object is created. It runs automatically whenever you instantiate a class using its name. You can think of the constructor as a setup function that prepares the new object with the right initial values. Defining your attributes in the constructor helps keep all objects of a class consistent and ensures they start with the required data.

```
class Car:
    def __init__(self, make : str, model : str):
        self.make = make
        self.model = model

    def display_info(self):
        print(f"This car is a {self.make} {self.model}.")
```

By defining the constructor, you give every new Car object a make and model immediately when created. Without the constructor, you would have to assign these manually after creation, which is error-prone and less readable.

Note: In this code and some of the future, you will see we note types for the IDE. Again not fully required, it is just helpful for us programming and using the objects.

1.3 Use the class as an object

Once you have the class created you will need to know how to use it. Remember the class itself is the blueprint for how an object works. When we create the object itself we call them "instances."

To create an instance of the object we simply use the class name followed by parenthesis and the arguments to send to it. Here we have two cars with different instance values. Then use the dot '`.`' operator to call the methods on it.

```
car1 = Car("Toyota", "Camry")
car2 = Car("Honda", "Civic")

car1.display_info()
car2.display_info()
```

2. Methods for Classes

You may be thinking what is this "method" I am reading about? A method is a function that belongs to a class and operates on the data of that specific object. In past chapters, we have been creating functions, as they don't belong to any object.

2.1 self parameter

When looking at any method in an object it will always have at least one parameter. The first parameter of any method in a class must be named 'self'. This parameter represents the current instance of the class — it's how the method accesses the object's own variables.

Note that when actually calling the method on an object, you don't send the 'self' reference yourself, it does it automatically.

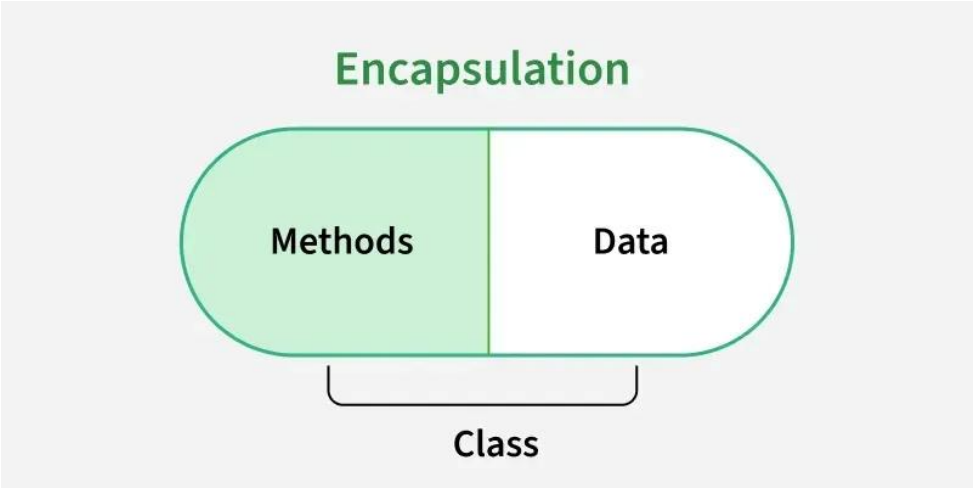
```
class Circle:
    def __init__(self, radius : float):
        self.radius = radius

    def area(self) -> float:
        return 3.14 * self.radius ** 2

c1 = Circle(5) # 5 is an int, but will be casted to float
print(c1.area())   # Equivalent to Circle.area(c1)
```

2.2 Encapsulation

One of the main features of using a class for OOP is that it helps us encapsulate all the data and behavior(methods) together inside an object and controlling access to that data.



2.3 Data Abstraction

Abstraction means hiding internal details and exposing only what's necessary to the outside world. This means that we only want people to call the methods on the object, not directly accessing the data members. Methods will help us better control our object. For example on a bank account, when depositing money you don't want someone to deposit a negative value, therefore adding a negative number to an account balance. Methods allow us to control these things.

Variable and Method access

Unlike other languages, we can't fully force Python to hide data, instead we follow some naming conventions for our variables and methods.

- Variables and methods starting with no underscores (`self.name`) are **public** – accessible anywhere
- Variables and methods starting with a single underscore (`self._name`) are **protected** – accessible within the class and its subclasses (more on subclasses next chapter)
- Variables and methods starting with two underscore (`self.__name`) are **private** – accessible only within the class.

```
class Employee:
    def __init__(self, name : str, salary : float):
        self.name = name
        self._department = "HR"
        self.__salary = salary

    def show_info(self):
        print(f"Name: {self.name}, Department: {self._department}")

emp = Employee("Alice", 50000)
emp.show_info()
print(emp.name)
# print(emp.__salary)  # Error
print(emp._Employee__salary)  # Not recommended
```

No actual errors will get thrown like other languages with C++, C# and Java. We still want to follow its access.

2.4 Getters and Setters

Since private variables can't be accessed directly, we use getter and setter methods to safely retrieve and modify them.

Below is a simple class for a bank account. Here you can see the get_balance() that just returns the private value of balance. We also have set_balance() that takes a value to change the balance only if the value sent is greater than or equal to 0. This controls the changing of values so that they don't get set to negative.

```
class Account:
    def __init__(self, owner : str, balance : float):
        self.__owner = owner
        self.__balance = balance

    def get_balance(self) -> float:
        return self.__balance

    def set_balance(self, amount : float):
        if amount >= 0:
            self.__balance = amount
            return True
        else:
            return False

acc = Account("John", 1000)
if acc.set_balance(1500):
     print("Account updated. Balance:", acc.get_balance())
else:
     print("Error, can't change balance.")
```

2.4.1 Property Decorators

Every language has its nuances or even tricks to help coding objects. Here in Python we can use what we call property decorators. Calling the `get_balance()` or `set_balance()` repeatedly can make code look clunky or slow you down in your coding process.

Using property decorators makes the syntax cleaner and allows for validation while maintaining easy access syntax.

```
class Account:
    def __init__(self, owner : str, balance : float):
        self.__owner = owner
        self.__balance = balance

    @property
    def balance(self) -> float:
        return self.__balance

    @balance.setter
    def balance(self, amount : float):
        if amount >= 0:
            self.__balance = amount

acc = Account("John", 1000)
acc.balance = 1500:
print(acc.balance)
```

Note, we can't really use true/false here since we are assigning values. But modifying our code to just modify will work.

2.4.2 Calling Setters from the Constructor

You will notice in the setters above for certain properties, like balance, we use an if statement to lock down and make sure we have values larger than zero. But we don't do anything like that in the constructor. That is a gap in the system to where we could potentially have negative values upon construction.

So you can simply code in an if statement in the constructor to solve this, but then we would have extra code. The process only expands if we have more variables. Instead, since it is already coded in the setter, we can actually easily call the setter from the constructor, fixing this problem for us.

```
class Account:
    def __init__(self, owner : str, balance : float):
        self.__owner = owner
        self.balance = balance

    @property
    def balance(self) -> float:
        return self.__balance

    @balance.setter
    def balance(self, amount : float):
        if amount >= 0:
            self.__balance = amount
```

You will see we are calling "**`self.balance`**" in the constructor but it isn't private. That is because just like in our other code, we are using the decorator that automatically calls the setter for us

Here is an example of not using the property decorators.

```
class Account:
    def __init__(self, owner : str, balance : float):
        self.__owner = owner
        self.set_balance(balance)

    def get_balance(self) -> float:
        return self.__balance

    def set_balance(self, amount : float):
        if amount >= 0:
            self.__balance = amount
```

2.5 Class vs Instance Variables

In all the examples so far, we have been using instance variables. These are variables we define in the constructor that are unique to each instance(object) of a class. Each object maintains its own copy of instance variable independent of other objects. A key reason for having 'self' in all method parameters.

There are times when we want to create a variable that is shared amongst all instances of a class. These are what we call a class variable and is defined at such level, outside of any method. All objects of the class share the same value for a class variable unless explicitly overridden in an object.

Here the variable "type" is a class variable.

```
class Car:
    type = "sedan"
    def __init__(self, make : str, model : str):
        self.make = make
        self.model = model

    def display_info(self):
        print(f"This car is a {self.make} {self.model} {type}.")
```

2.6 Other Programmer Defined Methods

We have talked about basic getter and setters with our class, but you can define whatever methods needed to help make your objects complete.

If we look back at our account, thinking if this is for say a bank, what other operations can we perform? I can think of things like "debit" for paying with a card, "credit" to give money back (like if you return something to store). All can be specific methods created on classes to perform set operations in the system being built.

```
class Account:
    interest = .005 # .5% interest

    def __init__(self, owner : str, balance : float):
        self.__owner = owner
        self.balance = balance

    @property
    def balance(self) -> float:
        return self.__balance

    @balance.setter
    def balance(self, amount : float):
        if amount >= 0:
            self.__balance = amount

    def debit(self, amount : float):
        if self.balance > amount: #make sure we have enough
            self.balance -= amount

    def credit(self, amount : float):
        if amount > 0: #add to balance if positive
            self.balance += amount

    # get our earned interest each month
    def apply_monthly_interest(self):
        self.__balance *= (1 + interest);
```

3. Magic Methods

Python has built in methods that can be overloaded to make operators, comparison and built-in functions like print() and len() work with our objects. Overloading (defining our own version) overwrites the generic built in methods and makes our objects more custom.

All of these "magic methods" can also be referred to as dunder methods, as they all start end with **d**ouble **under**scores.

- __init__ - of course already discussed, this is the constructor.
- __str__ - this defines a string value when str() is called on object or using print(). (Note: the print() calls the str() function internally)
- __len__ - defines what gets returned when the len() function is called.
- Comparison – send the object to compare, compares the instance variables
    - __eq__ - equality ==
    - __ne__ - not equal !=
    - __lt__ - less than <
    - __le__ - less than equal to <=
    - __gt__ - greater than >
    - __ge__ - greater than equal to >=
- Math operators – send the object to add, returns the adding of current instance to the other object instance
    - __add__
    - __sub__
    - __mul__
    - __truediv__
- __dict__ - returns a dictionary representation of the object. This can be helpful when dealing with JSON data on APIs.

A super important dunder method above that I would make sure to have in all my objects is the equals method **__eq__**. This method is used to compare similar objects and compares the variables you choose to compare. So, if you had two people in a user management system you might just compare names. But people can have the same names. Ok maybe their mailing addresses? Or checking emails? Or if you have the information a social security number/other identification number.

The **__eq__** method also gets called when using the "in" operator in Python. So that would be great to have a that functionality when checking if an object is in a list, or even in a dictionary (if you are using a class as the key). This saves on you coding a for loop, loop through the list and use the getter to check information

```python
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f"{self.title} ({self.pages} pages)"

    def __len__(self):
        return self.pages

    def __add__(self, other):
        return self.pages + other.pages

    def __eq__(self, other):
        return self.pages == other.pages

    def __lt__(self, other):
        return self.pages < other.pages

b1 = Book("Python Basics", 300)
b2 = Book("Advanced Python", 450)
b3 = Book("Python Basics", 300)

print(b1)
print(len(b1))
print(b1 + b2)
print(b1 == b3)
print(b1 < b2)
```

This example shows how you can customize an object's behavior. When you call print(b1), the __str__ method runs. The len() function calls __len__. The addition operator (+) uses __add__, and the equality and less-than comparisons use __eq__ and __lt__. By defining these methods, your class integrates smoothly with Python's built-in features.

4. Has-A Relationship (Composition)

Objects often contain or use other objects to model real-world relationships. A 'has-a' relationship (composition) means that one class includes another class as one of its attributes. This helps break programs into smaller, modular components that can be tested and reused independently.

```
class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

    def start(self):
        print(f"Engine with {self.horsepower} HP started.")

class Car:
    def __init__(self, make, model, horsepower):
        self.make = make
        self.model = model
        self.engine = Engine(horsepower)

    def drive(self):
        self.engine.start()
        print(f"{self.make} {self.model} is now driving!")

car = Car("Ford", "Mustang", 450)
car.drive()
```

Here, the Car class includes an Engine object. When the Car is created, its Engine is also created. This makes each part responsible for its own behavior — the Engine knows how to start, while the Car knows how to drive. Composition helps keep each class simple and focused on a single responsibility.

5. OOP Designing

The following are some tips, tricks and ways to thinking when doing Object Oriented Programming. By means am I a master at OOP. More experience will come as you do it more often.

1. Identify Nouns and Verbs

- Nouns become potential classes or attributes.
- Verbs become methods or actions for those classes.
- Example: Library system → nouns: Book, Member; verbs: borrow, return.

2. Distinguish 'Is-A' and 'Has-A' Relationships

- 'Is-A' → Inheritance (Dog is an Animal). *More on this next chapter.*
- 'Has-A' → Composition (Car has an Engine).
- Prefer composition unless shared logic truly requires inheritance.

3. Start Simple, Add Details Gradually

- Begin with 2–3 core classes and expand as needs arise.
- Each class should have one clear purpose (Single Responsibility Principle).

4. Organize Classes Across Modules Thoughtfully

- Keep each class or group of related classes in logical modules.
    - Small systems can hold multiple simple classes in one file.
    - Larger systems benefit from splitting each major class into its own module
- Group be relationship or responsibility, not size.
    - Classes that directly collaborate (e.g., Book, Library, Member) can stay in the same module.
    - Classes that serve different purposes (e.g., DatabaseManager, EmailNotifier) belong in separate files.
- Benefits
    - Easier navigation and maintenance.
    - Reduces merge conflicts in collaborative environments.
    - Allows selective importing (e.g., from library import Book, Member).

5. Model Object Interactions

- Determine which class 'controls' the workflow.
- Example: Library manages book checkouts, not Book itself.

6. Represent Real-World Data as Attributes

- Convert properties like color, speed, or cost into attributes.
- Keep them private when possible and access via getters/setters.

7. Assign Behavior to the Right Object

- Ask: Who has enough information to perform this action?
- Example: Engine knows how to start; Car calls Engine.start().

8. Focus on Behavior, Not Just Data

- Avoid 'data-only' classes; give them useful methods.

9. Use Encapsulation for Safety

- Make attributes private (__) and expose safe access methods.
- Prevent invalid data changes, like negative balances or ages.

10. Leverage Magic Methods

- __str__, __add__, and __eq__ make objects behave naturally.

11. Test with Real Scenarios

- Create example objects and test if interactions make sense.
- Revise if the workflow feels unnatural or redundant.

12. Abstract Real-World Details

- Focus on what matters for the program, not physical accuracy.
- Example: Building an HR system that tracks employees, clocking in/out and paychecks. Employees are people, we don't need behaviors for them to "eat food" but instead they need to "clock in" "clock out" or "calculate paycheck."

13. Think in Terms of Systems

- Classes should collaborate, not exist in isolation.
- Example: LibrarySystem manages Libraries, Members, and Books.

14. Sketch Diagrams

- Draw class boxes with arrows showing relationships.
- Helps catch redundancy or missing connections before coding.

15. Refactor Often

- Review designs as your program grows.
- Combine redundant classes or split large ones for clarity.

Practice Exercises

1) Create a Student Class – Build on what we covered with constructors and string methods:

- Include name, major, and GPA.
- Add a __str__ method that returns a formatted string like 'Name (Major) – GPA: x.x'.
- Test your class by creating a few students and printing their info.

2) Encapsulation Practice – Reinforce hiding data using getters and setters:

- Create an Account class with a private balance attribute.
- Write deposit() and withdraw() methods that modify the balance safely.
- Prevent negative balances or invalid withdrawals.

3) Has-A Relationship – Practice composition:

- Create a Library class that has a list of Book objects.
- Add methods to add, remove, and list books.
- Demonstrate the relationship by adding several books to a library.

4) Overloading Operations – Extend your objects with operator behavior:

- Build a Vector class that supports addition (__add__), subtraction (__sub__), and equality (__eq__).
- Add a __str__ method for a clean printout.
- Test by adding and comparing vectors.

5) Constructor and Getters – Use property decorators for validation:

- Create a Person class with name and age.
- Ensure age cannot be negative using a property setter.
- Include a __str__ method to display formatted details.